Summary of Terraform Associate Exam Preparation bullet points

PART 1:

1. Understanding Providers & Initialization

- Providers define how Terraform interacts with cloud platforms (e.g., AWS, Azure).
- After defining a provider, **`terraform init`** is required to initialize the working directory and download necessary plugins.
- Provider plugins are stored in the `.terraform` directory under `registry.terraform.io/hashicorp/aws`.

2. Is the Provider Block Mandatory?

- **No**, the provider block is not mandatory **if it is empty** (i.e., no specific configurations like region or credentials).
- Terraform can infer the required provider through the `required_providers` block.
- However, **a provider itself is mandatory** for creating resources.

3. Using Provider Aliases

- Aliases allow multiple configurations of the same provider for different regions or accounts.
- Example:

```
provider "aws" {
  region = "ap-southeast-1"
}

provider "aws" {
  alias  = "mumbai"
  region = "ap-south-1"
}

resource "aws_instance" "example" {
  provider = aws.mumbai
}
```

4. Best Practices for Credentials

- Storing credentials **inside the provider block** is not recommended.
- Use **environment variables** or external authentication mechanisms instead.

5. Terraform Settings (`terraform` Block)

- Contains:
  - **`required_version`**: Specifies Terraform version (e.g., `>= 1.2.0`).
  - **`required_providers`**: Defines provider dependencies and versions.
  - **Backend Configuration**: To be discussed in later sessions.

6. Key Terraform Commands

- **`terraform init`** → Initializes working directory and downloads providers.
- **`terraform init -upgrade`** → Upgrades provider versions based on constraints.
- **`terraform plan`** → Creates an execution plan but does **not modify state**.
- **`terraform apply`** → Applies changes and modifies the state file.
- **`terraform destroy`** → Removes Terraform-managed infrastructure.
- **`terraform fmt`** → Formats configuration files.
  - `terraform fmt -check` → Checks formatting without modifying files.
  - `terraform fmt -recursive` → Formats files in subdirectories.

7. Version Constraints & Provider Tiers

- Version constraints use operators like `>=`, `<=`, `~>`.
- Providers are categorized into:
  - **Official** (maintained by HashiCorp)
  - **Partner** (verified by third-party vendors)
  - **Community** (custom providers)

8. State File & Terraform Behavior

- The default Terraform state file is **`terraform.tfstate`**.
- Terraform **plan does not modify** the state file.
- **Destroying infrastructure** can be done either via:
  - **`terraform destroy`**
  - **Removing a resource from `.tf` files** and running `terraform apply`.

PART 2:

1. Terraform Validate (`terraform validate`)

- Validates Terraform configuration files in the directory.
- Requires an **initialized directory** (`terraform init` must be run first).
- **`terraform plan` also includes an implicit validation check**, so `terraform validate` is not always necessary.

2. Terraform Refresh (`terraform refresh`)

- Reads current settings of managed resources and **updates the state file** to match them.
- **Deprecated** due to unsafe behavior (replaced by `terraform apply -refresh-only`).

3. Resource Block Structure

- **Resource Type**: `aws_instance`
- **Local Name**: `web`
- **Arguments**: `ami`, `instance_type`
- **Example Resource Address**: `aws_instance.web`
- **Resource names must be unique within a module.**

4. Terraform Data Types

- **Supported Types**: `string`, `number`, `boolean`, `list`, `set`, `map`, `null`.
- **Terraform does not support arrays**.

5. Terraform State Management

- Important `terraform state` subcommands:
  - **`terraform state move`** → Moves resources within or across state files.
  - **`terraform state remove`** → Removes a resource from state without deleting it in the cloud.

6. Debugging Terraform

- Enable logs using environment variables:
  - **`TF_LOG`** (`TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR`)
  - **`TF_LOG_PATH`** (Saves logs to a file)

7. Terraform Import (`terraform import`)

- Imports existing infrastructure into Terraform.
- **New feature: Automatic code generation** for imported resources.
- Supports **multiple resource imports using `import` blocks**.

8. Local Values (`locals`)

- Used to **avoid repetition** of the same expression multiple times.
- Defined inside a **`locals` block** but referenced using **`local.`** (singular).

9. Terraform Workspaces

- Allows managing multiple deployments from the **same configuration**.
- State files are stored in **`terraform.tfstate.d`**.
- **Not suitable for strong isolation** (e.g., separate staging and production environments).
- Common workspace commands:
  - `terraform workspace new <name>`
  - `terraform workspace select <name>`
  - `terraform workspace list`

10. Terraform Modules

- **Encapsulates reusable infrastructure code**.
- Module locations:
  - **Local filesystem** (`./modules/example`)
  - **Git repositories** (`git::https://github.com/example.git`)
  - **Terraform Registry** (`terraform-aws-modules/vpc/aws`)
- **Root Module vs. Child Module**:
  - **Root module** is the main entry point.
  - **Child module** is any module **called** by another module.
- **Module Outputs** expose values to other modules.
- **Versioning**:
  - **Not mandatory** but recommended (`version = "~> 1.2"`).

11. Terraform Registry

- **Hosts publicly available Terraform modules**.
- Module address format: `[hostname]/[namespace]/[name]/[system]`
- **Default registry**: `registry.terraform.io`

12. Terraform Functions

- **Terraform does not support user-defined functions**.
- Important built-in functions:
  - **`lookup()`** → Retrieves values from a map.
  - **`zipmap()`** → Converts lists into a map.
  - **`index()`** → Returns index of an element in a list.
  - **`element()`** → Retrieves element at a specific index.
  - **`toset()`** → Converts a list into a set.
  - **`timestamp()`** → Retrieves current timestamp.
  - **`file()`** → Reads content from a file.

13. Meta-Arguments in Terraform

- **Meta-arguments allow customizations in resource blocks**.
- **Common meta-arguments:**
  - **`depends_on`** → Specifies dependencies between resources.
  - **`count`** → Creates multiple instances of a resource.
  - **`for_each`** → Iterates over a map or set.
  - **`lifecycle`**:
    - `ignore_changes` → Ignores changes to specified attributes.
    - `prevent_destroy` → Prevents accidental deletion.
    - `create_before_destroy` → Ensures new resources are created before old ones are destroyed.

PART 3:

1. Terraform Sentinel (Policy as Code)

- Sentinel is **a policy as code framework** used to enforce security and compliance rules.
- Runs **after `terraform plan` but before `terraform apply`**.
- Use cases:
  - Enforcing **tagging policies** for EC2 instances.
  - Checking if an **S3 bucket is encrypted**.

---

2. Terraform Graph (`terraform graph`)

- **Visual representation of resource dependencies**.
- Generates output in **DOT format**, which can be converted into an image.

---

3. Input Variables & tfvars Files

- **`terraform.tfvars`** allows multi-environment deployments (e.g., Dev, Prod).
- Select a specific variable file using:

```
terraform plan -var-file="prod.tfvars"
```

- **Variable precedence (last one wins):**
  1. **Environment variables** (`TF_VAR_` prefix).
  2. `terraform.tfvars` or `.tfvars.json` files.
  3. `auto.tfvars` or `.auto.tfvars.json` files.
  4. **Command-line variables** (`-var` and `-var-file`).

- Reserved variable names that **cannot be used**:
  `count`, `depends_on`, `for_each`, `lifecycle`, `providers`, `source`.

---

4. Terraform State & Output Values

- **State file (`terraform.tfstate`) does not store input variables**.
- **Output values** are stored in the state file.
- Use **`terraform output`** to extract values from the state file.

---

5. Terraform Console (`terraform console`)

- Used for testing Terraform expressions and functions interactively.

---

6. Dependency Lock File (`terraform.lock.`)

- **Locks provider versions** to avoid unexpected upgrades.
- **`terraform init -upgrade`** updates the lock file.

---

7. Dependencies in Terraform

- **Implicit dependency** → Terraform **automatically** determines the order.

```
resource "aws_eip" "example" {
  instance = aws_instance.myec2.id  # Implicit dependency
}
```

- **Explicit dependency** → Manually define order using `depends_on`.

```
resource "aws_eip" "example" {
  depends_on = [aws_instance.myec2]
}
```

---

8. Terraform Data Sources

- Fetches and **uses data from external resources** (e.g., existing AWS VPCs).

---

9. Terraform Enterprise & HCP Terraform

- Terraform **Enterprise & HCP** provide:
  - **Single sign-on (SSO)**
  - **Audit logging**
  - **Team management & governance**
  - **Private Module Registry**
  - **Remote state storage with encryption**
- Terraform Cloud (HCP) **automatically runs plans when code is pushed to VCS**.

---

10. Protecting Secrets in HCP Terraform

- **Mark Terraform or environment variables as "Sensitive"** to prevent visibility.
- Sensitive values **cannot be viewed even by admins**.

---

11. Recreating Resources in Terraform

- **Force resource replacement** using:

```
terraform apply -replace="aws_instance.example"
```

- **`terraform taint` is deprecated**; use `-replace` instead.

---

12. Infrastructure as Code (IaC) Benefits

- **Automation**
- **Versioning**
- **Reusability**
- Common IaC tools:
  - Terraform (multi-cloud)
  - AWS CloudFormation
  - Azure Resource Manager
  - Google Cloud Deployment Manager

---

13. Terraform Output Command (`terraform output`)

- Extracts values from **output variables** stored in the state file.

---

14. Terraform & Git

- **By default, Terraform pulls from the default branch in Git.**
- To specify a branch or commit:

```
source = "git::https://github.com/example.git?ref=branch-name"
```

---

15. Terraform Splat Expressions (`[*]`)

- **Shorter syntax for accessing attributes in a list.**

```
aws_instance.example[*].id
```

- **Incorrect Syntax:**

```
aws_instance.example.*.id  # Legacy syntax
```

- **Wrong usage with dot (`.`)** will cause errors.

---

16. Fetching Values from Lists & Maps

- **Fetching list values:**

```
var.size[1]  # Retrieves the second item (indexing starts at 0)
```

- **Fetching map values:**

```
var.type["ap-south-1"]
```

---

17. Handling Large Infrastructures

- **Cloud providers enforce API rate limits** → Split configurations.
- Use:

```
terraform apply -refresh=false -target=resource_name
```

---

18. Terraform Backends

- Default: **Local backend** (stores state on disk).
- Remote backends:
  - **S3**
  - **Azure Storage**
  - **Terraform Cloud**
- Migrating state to a new backend triggers **automatic migration**.

---

19. Terraform Enterprise Air Gap Installation

- **Used for environments without internet access.**
- Supports **offline provider/plugin installation**.

---

20. Publishing Modules to Terraform Registry

- **Requirements:**
  - Must be hosted on **GitHub**.
  - Naming: `terraform-<PROVIDER>-<NAME>`.
  - Follow standard **module structure**.
  - **Use version tags** (e.g., `1.0.0`).

---

21. Comments in Terraform

- **Single-line:** `#` or `//`
- **Multi-line:** `/* comment */`

---

22. Terraform Dynamic Blocks

- **Used for generating nested configurations dynamically.**
- Overuse can make code harder to read.

---

23. Miscellaneous Pointers

- **GitHub is NOT a supported Terraform backend.**
- **Terraform providers can be installed offline**.
- **Terraform does NOT require Go** as a prerequisite.
- **Terraform and provider versions do not have to match.**

---

24. Handling State Locking

- **Terraform automatically locks state during modifications**.
- **State file locking prevents:**
  - `terraform apply`
  - `terraform refresh`
  - `terraform state rm/mv/push`
- **To manually unlock state:**

```
terraform force-unlock <LOCK_ID>
```

---

25. State File Security

- **State file may contain sensitive data** → Store in **remote backends**.
- Use `.gitignore` to exclude:

```
terraform.tfstate
terraform.tfvars
```